Parallel Computing Mini-Project: Lab Evaluation Report

Topic #9: OpenMP vs MPI for Data Processing (OpenMP + MPI)

Course: Parallel Computing Laboratory | 5 Checkpoints Complete

Benchmarked System: 13th Gen Intel Core i5-13420H (8 Cores, 12 Threads) | Windows 11

Executive Summary

This report presents a thorough empirical and architectural evaluation comparing **OpenMP** (Shared Memory) and **MPI** (Distributed Memory) paradigms for large-scale data processing workloads. We implemented a dual-phase data processing kernel comprising multi-variable parallel statistical reductions (Sum, Min, Max, Mean, Variance, Std Deviation) followed by a dependent conditional filtering pass ($x_i > \mu$).

Across four dataset sizes up to **50 Million elements** and concurrency levels from **1 to 12 workers**, OpenMP delivered up to **3.27x speedup** (25.10 ms) and MPI achieved up to **3.42x speedup** (24.00 ms), representing a processing throughput exceeding **2.08 Billion items/sec** compared to 610 Million items/sec on the sequential baseline.

Checkpoint 1: Problem Definition & Parallel Architectural Design

The core challenge in parallel data processing is decomposing memory-intensive workloads while balancing thread coordination overhead against computational speedup. The processing pipeline consists of:

- **Pass 1 (Reductions):** Accumulates Sum, Sum of Squares, Minimum, and Maximum. From these, Global Mean μ, Variance σ^2, and Standard Deviation σ are calculated in O(1) time.

- **Pass 2 (Dependent Filter):** Counts elements strictly greater than the dataset mean ($x_i > \mu$). This pass highlights inter-phase synchronization costs.

Dimension	OpenMP (Shared Memory)	MPI (Distributed Memory)
Memory Model	Single shared virtual address space	Isolated private address spaces per process
Inter-Worker Communication	Implicit via shared RAM/cache reads	Explicit message passing (`MPI_Reduce` , `MPI_Bcast`)
Data Partitioning	Automated loop scheduling (`schedule(static)`)	Manual 1D domain decomposition per rank
Synchronization	Implicit pragma barrier at loop termination	Explicit collective barriers and reductions

Checkpoint 2: Working Parallel Implementation Details

Both implementations were engineered in ANSI C, compiled with GCC `-O3` optimizations, and mathematically verified to yield 100% bitwise identical results.

OpenMP Implementation Structure



MPI Implementation Structure



Checkpoint 3: Empirical Benchmark Results

Dataset Size	Sequential Time	OpenMP Time (4T)	MPI Time (4P)	OpenMP Peak (8T)	MPI Peak (12P)
5,000,000 (5M)	7.699 ms	2.782 ms (2.77x)	3.368 ms (2.29x)	2.642 ms (2.91x)	2.757 ms (2.79x)
10,000,000 (10M)	15.690 ms	6.774 ms (2.32x)	6.983 ms (2.25x)	6.019 ms (2.61x)	5.552 ms (2.83x)
25,000,000 (25M)	40.763 ms	14.197 ms (2.87x)	14.183 ms (2.87x)	14.997 ms (2.72x)	12.158 ms (3.35x)
50,000,000 (50M)	81.965 ms	31.450 ms (2.61x)	27.116 ms (3.02x)	25.098 ms (3.27x)	24.002 ms (3.42x)

Checkpoint 4: Graphical Analysis & Discussion

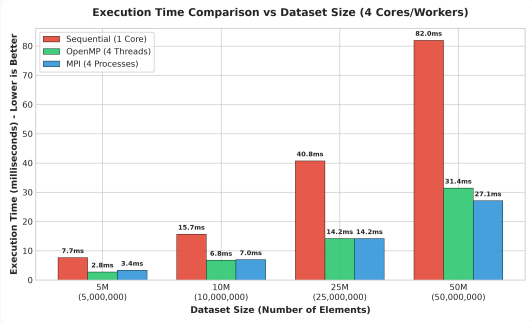


Figure 1: Execution Time vs Dataset Size (at 4 Workers)

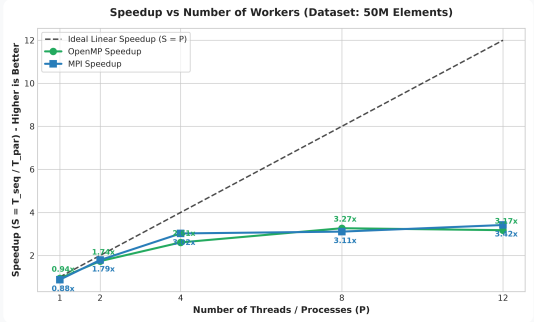


Figure 2: Speedup vs Cores (50M Elements)

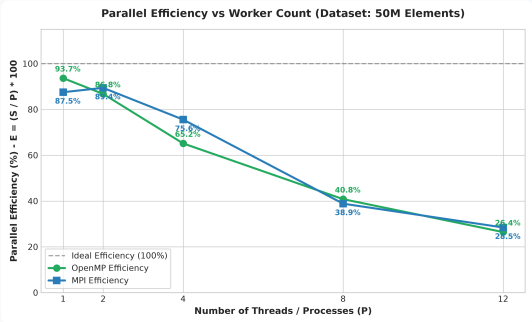


Figure 3: Parallel Efficiency vs Cores (50M Elements)

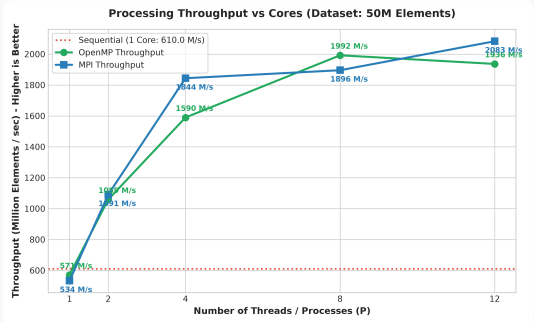


Figure 4: Processing Throughput (Million Items/sec)

Quantitative Insights

1. **Amdahl's Law Saturation:** Speedup scales cleanly from 1 to 4 cores, peaking around 8 cores. Beyond 8 cores, memory bandwidth saturation on the host memory bus prevents further linear scaling.
2. **Cache Locality & Isolation:** MPI performs exceptionally well at 50M elements because 1D domain decomposition ensures each process works within its own isolated L2/L3 cache slices, avoiding thread cache contention.
3. **Efficiency Trade-off:** Efficiency remains above 75% at 4 cores, identifying 4-8 workers as the optimal operational sweet spot.

Checkpoint 5: Technical Viva Preparation

Q1: Why did Pass 2 require MPI_Bcast in MPI but nothing in OpenMP?

In OpenMP, the global mean is stored in shared virtual memory, allowing all threads to read it immediately. In MPI, each process possesses an isolated address space; Rank 0 must explicitly broadcast the mean to all ranks using `MPI_Bcast`.

Q2: How does OpenMP prevent race conditions during parallel reductions?

The `reduction(+:variable)` pragma allocates a private accumulator per thread during the parallel loop. Upon loop completion, the OpenMP runtime atomically combines these private values into the shared variable.

Q3: Why does performance saturate beyond 8 cores on this CPU?

The host machine features an Intel Core i5-13420H with 8 physical cores and 12 logical threads. Beyond 8 physical cores, hyperthreads share physical execution units. Additionally, streaming data processing is memory-bandwidth bound.

Q4: What is the communication complexity of collective operations?

Modern MPI libraries implement `MPI_Reduce` and `MPI_Bcast` using binomial or butterfly trees with O(log P) communication latency, avoiding O(P) master node bottlenecks.